Alessio Rella
Professor Kambhampaty
CMPSC 132
28 April 2026

Tic-Tac-Toe Python Program

Project Description

I created a python program that starts a two player couch co-op game of tic-tac-toe in which players take turns marking a spot on a 3x3 grid until a winner or a draw is reached (as per usual tic-tac-toe rules) that runs in the game terminal.

The program starts by putting the user into a main menu, from which they can select to (1) play the game, (2) change the symbols they are playing with (which is 'X' and 'O' by default), (3) swap which player goes first, (4) see the current scoreboard, or (5) exit the program. The scoreboard keeps track of how many times player 1 wins, how many times player 2 wins, their current symbols, and how many times the game ended in a draw. After the user finishes a game, they have the option to play again, play again with the order of play swapped (a quality of life feature so one must not return to the main menu after every match), or return to the main menu.

Approach, Logic, and Data Structures Used

The first decision I made was how to display the board. I first decided to label the columns with numbers 1-3 and the rows with letters a-c, as shown below.

	1	2	3
A	A1	A2	A3
B	B1	B2	B3
C	C1	C2	C3

I then created a dictionary with each cell as a key, where the value is a single character long string. Each value starts as a space to display an empty board. As the game continues, the dictionary is updated, which is reflected in the string representation of the gameboard. (Board → __str__(self)).

I will now briefly discuss each class and function and what they do.

class Board → stores all information regarding the state of the game, including the turn count, the current state of the board, the turn order, the action of a player on their turn, and checks for if a win or draw condition has been met. It initializes with an empty board, a player order, turns set to 0, and draws set to 0.

clear_board(self) → resets the board to empty

get_pos(self, pos) → I wanted players to be able to input their cell selection in any format (e.g. 1a, A1, a 1, etc. are all viable inputs). get_pos takes an inputted position and (1) reformats it to match how the dictionary keys are formatted, and (2) verifies that it is a valid cell (i.e. the input only contains valid one number and valid one letter. It also assumes that if there are any miscellaneous symbols (aside from spaces), the input was entered incorrectly and raises an error.)

isEmpty(self, pos) → Checks that a position is empty (i.e. = ` `)
update_board(self, pos, sym) → updates the board dictionary with pos, sym as a key, value pair.
swap_turn_order(self) → One of the attributes of the class Board is players. Players is a list, where the order of the Players in that list determines the order of play. This function swaps that order.
check_winner(self) → There are 8 win cases in tic-tac-toe: 3 by row, 3 by column, and 2 by diagonal. To check these, I defined the rows as a list of lists, and the same for the columns and diagonals (e.g. rows = [['a1','a2','a3'],['b1','b2','b3'],['c1','c2','c3']]) This function iterates through each row, column, and diagonal, and checks if the associated value in the dictionary for each key is equivalent and not a space. If so, this is a win condition. This function returns which symbol won.
check_draw(self) → Check draw returns true if every cell is filled. It checks this by calling isEmpty(self,pos) for all positions.
play_game(self) → This handles the overall game. It has a while loop that stops when a win or draw condition is met. It alternates turns by checking if the turn count is odd or even. It then requests an input from the correct player. This input is checked (valid and empty cell), and then a call is made to take_turn(). After the game ends, play_game(self) clears the board and resets the turn counter, before giving the player a menu where they select from (1) play again, which makes a new call to play_game(), (2) play again with turn order swapped, or (3) return to main menu.
take_turn(self, pos, player) → This function calls update_board(), increments the turn counter, and checks if the turn resulted in a win or draw. If a player won, it increments their win count. It also prints the board after each turn is taken.

class Player → stores information regarding the two players. This being the number of wins, and the symbol that they are playing as.
update_sym(self, other) → This function is called from the main menu, and allows the players to change what symbols they are playing as. It ensures the selected symbols are one character in length (to keep the board lined up with itself), and that the symbols are different from each other. If these conditions are not met, players are asked to choose a new symbol.

main() → The main menu function just acts as a directory for players to use different functions without typing them into the terminal. Options are selected by entering a number. I chose to include this menu to keep the game accessible and as easy to start playing as possible.

The final four lines in the script just initialize the players and board, and put the user into the main menu.

<u>Challenges Faced and Possible Improvements</u>

The hardest part of this project for me was implementing the game in such a way that it didn't require each turn to be typed into the terminal itself. I think the main() function I implemented is a good way of allowing the user control over when the game starts and when they quit.

For improvements, update_sym(self, other) has a lot of repeated code for getting each of the inputs. I think play_game() and take_turn() are also both complicated in a way I'm not happy with. I tried to delegate tasks to functions, but these are both doing a lot. I also think it would have been cleaner to have play_game() ask for the players as inputs, instead of having the players attached to the board itself (e.g. play_game(self,player1,player2)). I found myself getting confused by the way I implemented it as I was working on the project.